



Regresión Lineal

Aprendizaje Automático 1

Gradiente Descendiente

Descent Gradient



Gradiente Descendiente

Es un algoritmo de optimización: busca los valores de los parámetros que hacen más chico el error del modelo.

Es iterativo. En vez de resolver una fórmula de una sola vez, se acerca de a pasos.

Con datasets grandes, iterar suele ser mucho más barato que invertir matrices.

Hoy vemos tres variantes: gradiente descendiente, estocástico y mini-batch.

Notación: quién es quién

Lo que inicializamos y ajustamos son los PARÁMETROS

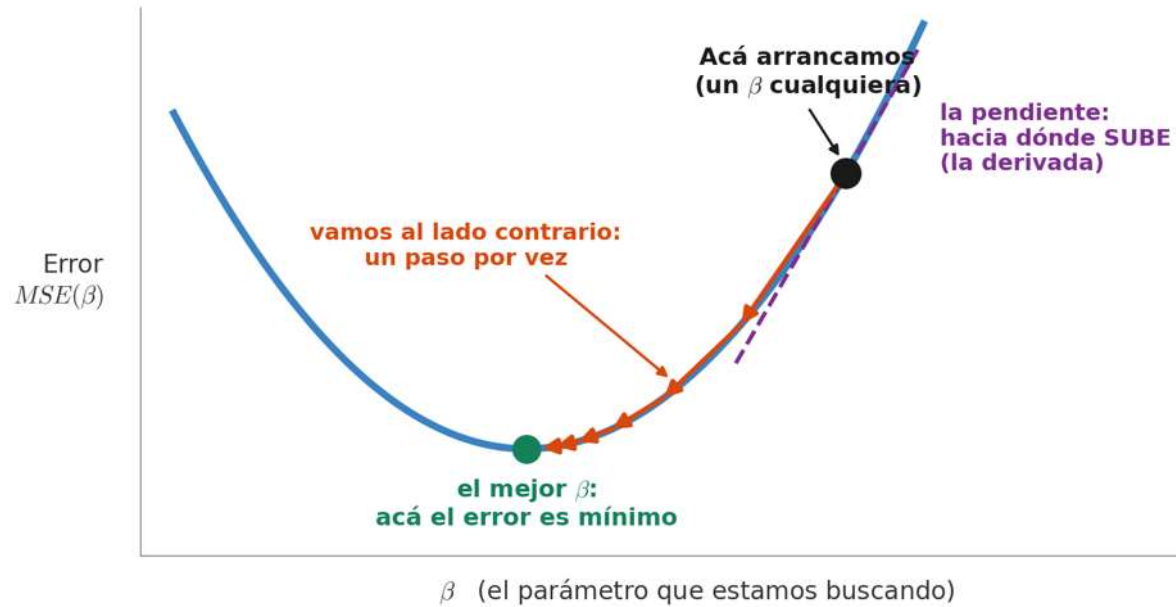
$$\beta = [\beta_0, \beta_1, \beta_2, \dots, \beta_p]$$

el intercepto:
dónde arranca

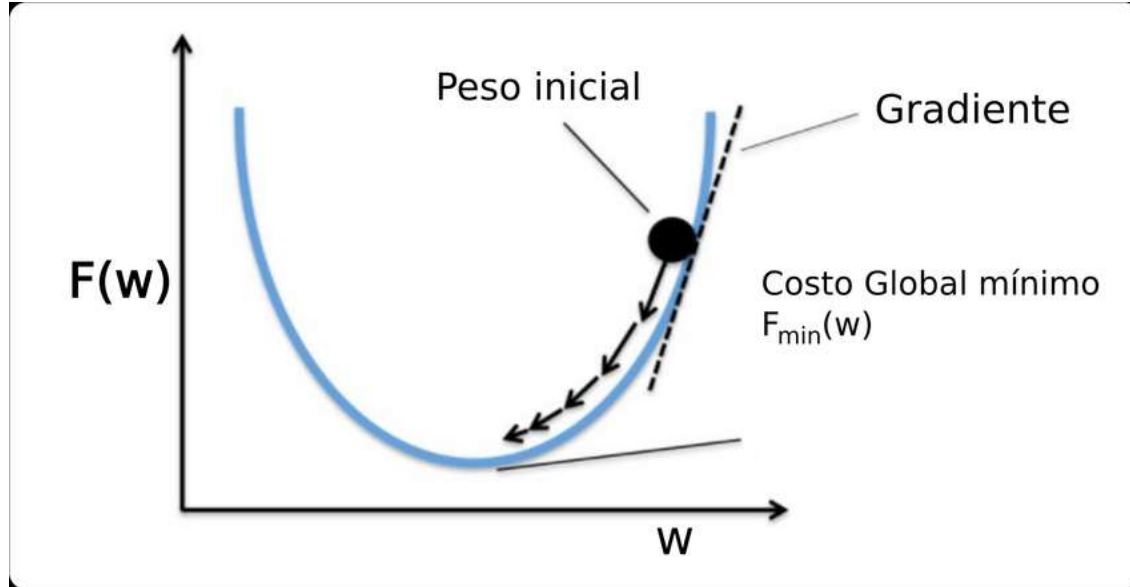
uno por cada característica:
cuánto pesa cada variable

Esto es lo único que el algoritmo va cambiando. Los datos (x, y) nunca se tocan.

Ejemplo



Generalizado



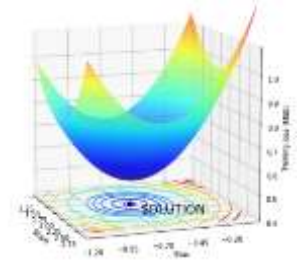
¿Qué es el gradiente?

El gradiente de una función es un vector que apunta en la dirección de la mayor derivada de la función en un punto dado. Establece la dirección en la que la función está aumentando más rápidamente.

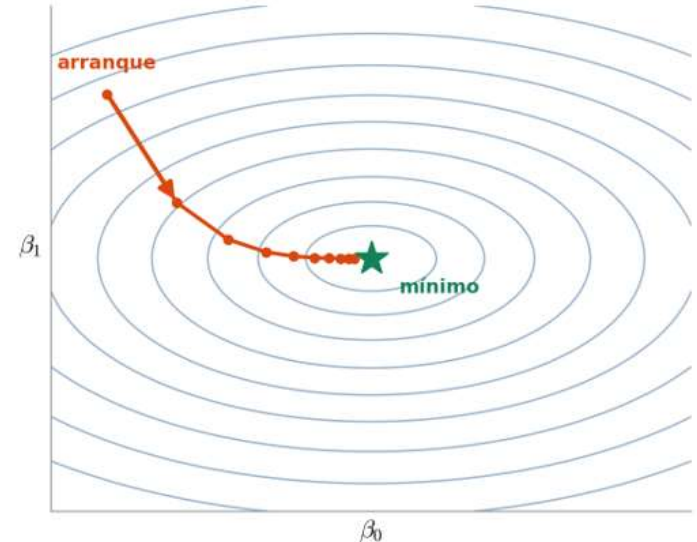
Cuando estamos tratando de minimizar una función, se busca encontrar el punto donde la función alcanza su valor mínimo. En el mínimo de la función, el gradiente es igual a cero.

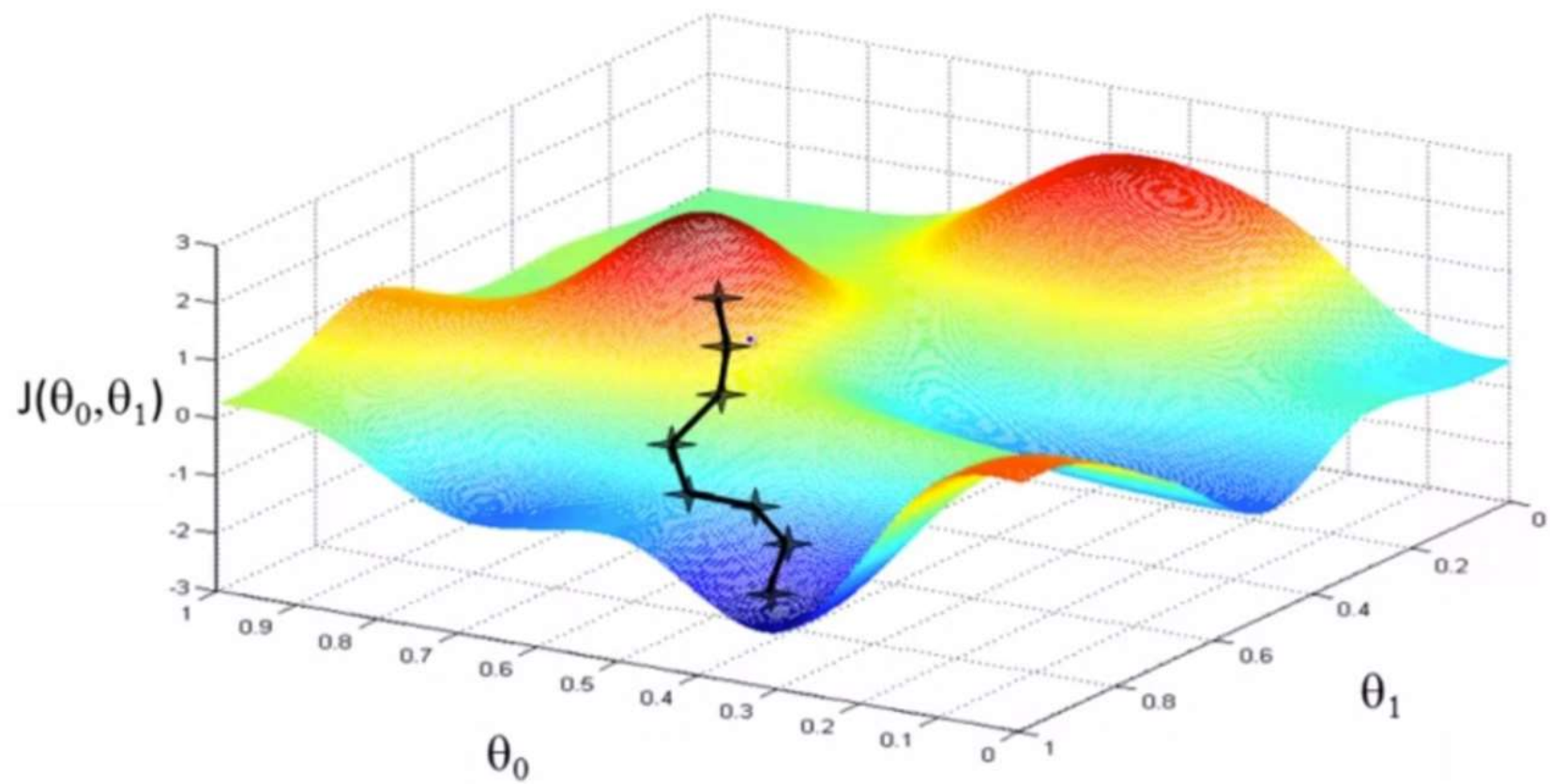
Si se mueve en la dirección opuesta al gradiente, nos acercamos al mínimo.

Pero puede ser un mínimo global, o local



El mismo valle, visto desde arriba
(como un mapa topográfico)







Técnica del gradiente descendiente

Se parte de un vector inicial en el espacio de n dimensiones (parámetros) $(b_1, b_2, \dots, b_n)$.

Se obtiene el gradiente de la función de pérdida (u optimización) para ese punto x .

$$\nabla MSE(\beta_0, \beta_1) = \left[\frac{\partial MSE}{\partial \beta_0}, \frac{\partial MSE}{\partial \beta_1} \right]$$

$$\frac{\partial MSE}{\partial \beta_0} = \frac{-2}{N} \sum_{i=1}^N (y_i - \beta_0 - \beta_1 \cdot x_i)$$

$$\frac{\partial MSE}{\partial \beta_1} = \frac{-2}{N} \sum_{i=1}^N x_i \cdot (y_i - \beta_0 - \beta_1 \cdot x_i)$$



Técnica del gradiente descendiente

$$\nabla MSE(\beta_0, \beta_1) = \left[\frac{-2}{N} \sum_{i=1}^N (y_i - \beta_0 - \beta_1 \cdot x_i), \frac{-2}{N} \sum_{i=1}^N x_i \cdot (y_i - \beta_0 - \beta_1 \cdot x_i) \right]$$

```
def gradient(X, y, beta0, beta1):  
    N = len(y)  
    y_hat = beta0 + beta1 * X  
  
    d_beta0 = (-2/N) * np.sum(y - y_hat)  
    d_beta1 = (-2/N) * np.sum(X * (y - y_hat))  
  
    return d_beta0, d_beta1
```

sería mejor de forma matricial para no tener que codear por cada feature del modelo, ¿no?



Técnica del gradiente descendiente

```
def gradient(X: np.ndarray, y: np.ndarray, theta: np.ndarray) -> np.ndarray:

    N = len(y)
    y_hat = X.dot(theta)

    grad = (-2 / N) * X.T.dot(y - y_hat)

    return grad
```

Técnica del gradiente descendiente

La regla que se repite una y otra vez

$$\beta_j^{\text{nuevo}} = \beta_j^{\text{viejo}} - \alpha \cdot \frac{\partial \text{MSE}}{\partial \beta_j}$$

Diagram illustrating the components of the gradient descent rule:

- el MENOS:** vamos al lado contrario (indicated by an orange arrow pointing down from the minus sign)
- learning rate:** tamaño del paso (indicated by a purple arrow pointing down from the Greek letter α)
- la pendiente:** hacia dónde SUBE el error (indicated by a blue arrow pointing down from the partial derivative term)



Técnica del gradiente descendiente

Se invierte la dirección del gradiente. Para ir hacia la dirección de minimización.

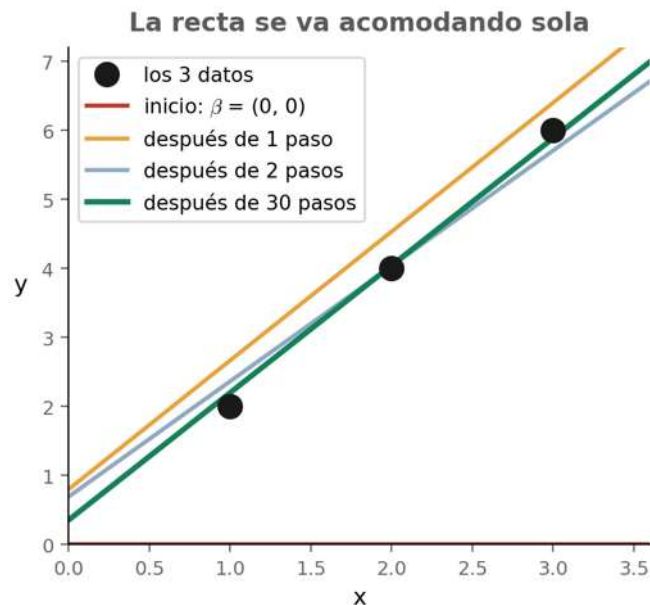
Se lo pondera con un factor de aprendizaje (usualmente llamado `learning_rate` o `lr`).

Se actualiza el vector

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

Se repite el proceso, hasta una cantidad máxima de iteraciones o hasta encontrar una tolerancia adecuada en el error.

Una iteración a mano



Datos: $x = (1, 2, 3)$ $y = (2, 4, 6)$

Arrancamos: $\beta_0 = 0, \beta_1 = 0, \alpha = 0,1$

Predicción: $\hat{y} = (0, 0, 0)$

Residuos: $y - \hat{y} = (2, 4, 6)$

Pendiente β_0 : $-\frac{2}{3}(2 + 4 + 6) = -8,00$

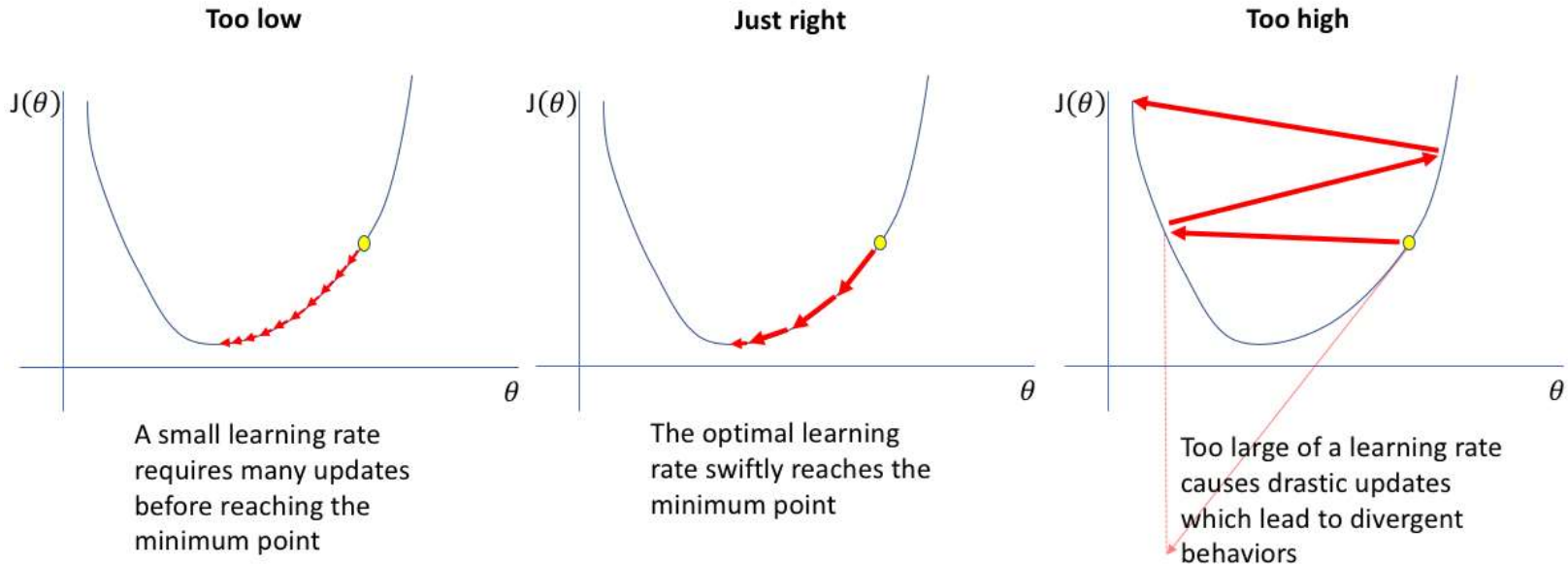
Pendiente β_1 : $-\frac{2}{3}(1 \cdot 2 + 2 \cdot 4 + 3 \cdot 6) = -18,67$

Actualizamos: $\beta_0 = 0 - 0,1 \cdot (-8,00) = 0,80$

$\beta_1 = 0 - 0,1 \cdot (-18,67) = 1,87$

Y se repite. El error pasó de 18,67 a 1,66 en un solo paso.

Learning rate (ratio de aprendizaje)





Gradiente descendiente estocástico



Gradiente descendiente estocástico

También busca optimizar por la dirección opuesta al gradiente.

La diferencia es que no se usa todo el dataset para calcular el gradiente, **sino de a un dato**.

Se ordenan los datos aleatoriamente. Se calcula el gradiente para la primer muestra, se actualiza el vector y se continúa así con todas las muestras. Recién ahí, se llega a la segunda época.

El enfoque estocástico se da al “muestrear” el dataset.



Qué ganamos y qué perdemos

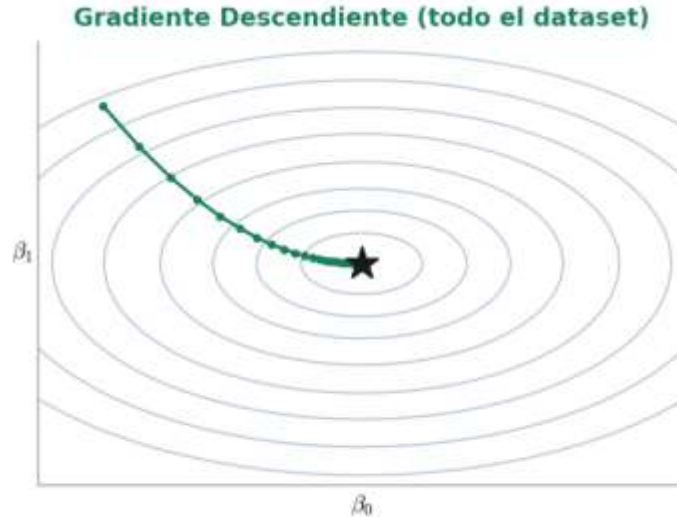
A favor: Calcular el gradiente con un solo dato es muchísimo más barato que hacerlo con el dataset completo.

A favor: Empieza a mejorar desde el primer dato, sin esperar a recorrer todo.

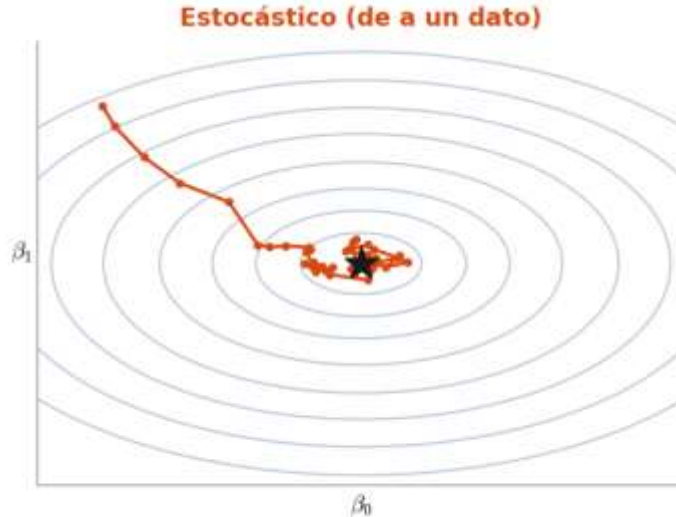
En contra: Con un solo dato la dirección es ruidosa. El camino zigzaguea y da vueltas alrededor del mínimo en vez de clavarlo.

En contra: En la práctica termina siendo lento.

GD vs SGD: el mismo destino



Camino suave y directo.
Cada paso cuesta caro,



Camino ruidoso, da vueltas cerca del mínimo.
Cada paso cuesta muy poco,



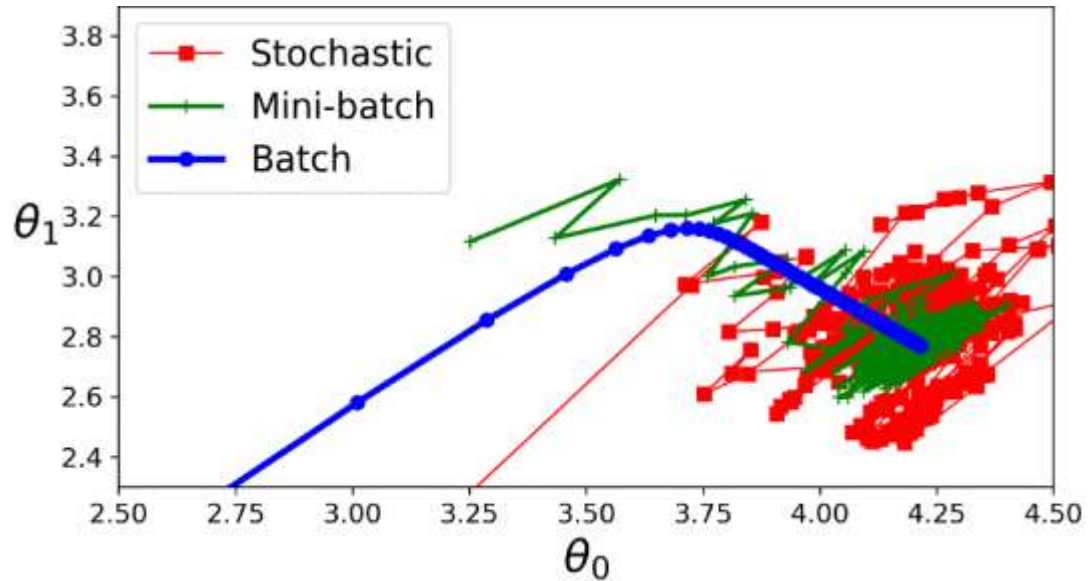
Gradiente descendiente mini-batch



Gradiente Descendiente mini-batch

- Ni usamos todo el dataset ni solo una muestra a la vez, sino un batch (lote).
- El dataset se divide en n batches.
- Se calcula el gradiente para un batch y se ajusta el vector. Luego para el siguiente batch y así sucesivamente.
- Luego de una época, se vuelven a obtener los distintos batches (para que tengan datos distintos).
- Converge más rápido que GD y con menor ruido que SGD.
- El tamaño del batch es un **hiperparámetro** que debe ajustarse: si es muy grande, es similar a GD, y si es muy chico, es similar a SGD.

GD vs SGD vs Mini-Batch GD



Resumen:

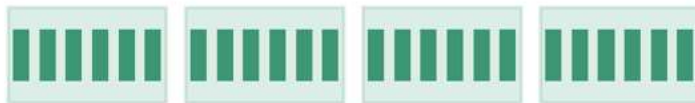
Cada barrita es un dato; cada recuadro, una actualización de β

Gradiente Descendiente



1 actualización por época

Mini-batch (lotes de 6)



4 actualizaciones por época

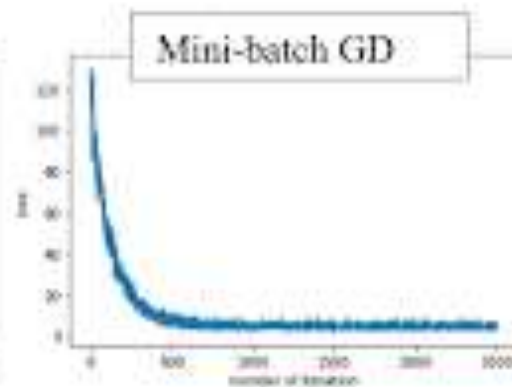
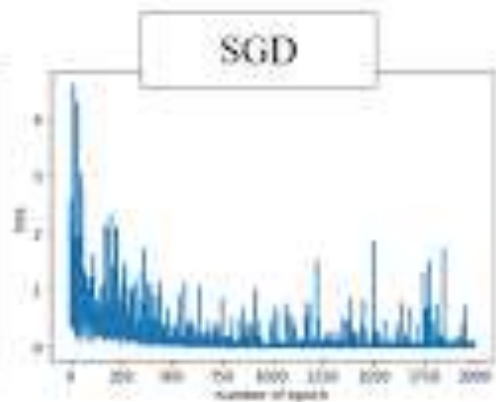
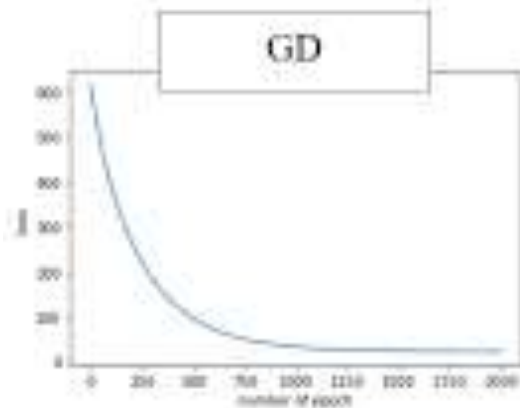
Estocástico (SGD)



24 actualizaciones por época

1 época = una pasada completa por TODOS los datos. Lo que cambia es cada cuánto actualizamos β .

GD vs SGD vs Mini-Batch GD





GD vs SGD vs Mini-Batch GD

	Gradient Descent	Stochastic Gradient Descent	Mini-Batch Gradient Descent
Gradient	$\nabla \left(\sum_{\text{all samples}} (y_i - f_W(X_i))^2 \right)$	$\nabla ((y_i - f_W(X_i))^2)$	$\nabla \left(\sum_{\text{batch samples}} (y_i - f_W(X_i))^2 \right)$
Speed	Very Fast (vectorized)	Slow (compute sample by sample)	Fast (vectorized)
Memory	O(dataset)	O(1)	O(batch)
Convergence	Needs more epochs	Needs less epochs	Middle point between GD and SGD
Gradient Stability	Smooth updates in params	Noisy updates in params	Middle point between GD and SGD